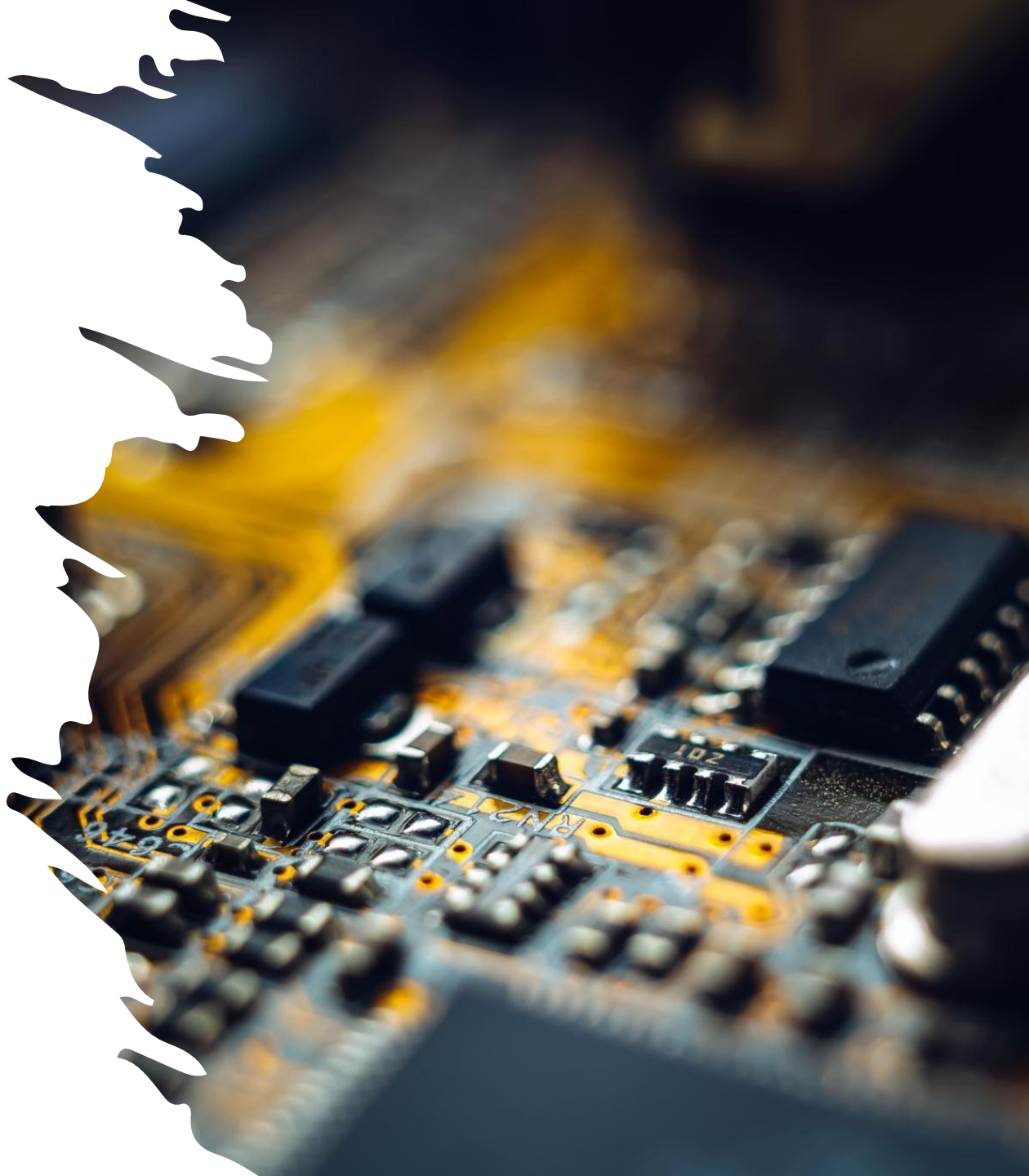
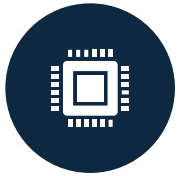
What would happen if every app on your phone could directly control the camera, memory, and CPU without asking permission?



More questions?



Who actually controls the CPU — the application or the operating system?

Can a program access hardware directly?

How does an application ask the OS to open a file?

Why can’t user programs run in full control mode?

What protects the system if an application crashes?

How does the OS decide what is allowed and what is not?

Operating System Services and Structure

By: Bilal Ahmed

Learning Objectives

Classify the services of an operating system.

Explore the tasks within services.

Analyze the need of the system calls.

Understand the purpose of system calls.

OS Services

Process Control

File System Manipulation

Input/Output (I/O) Operations

Resource Allocation

Inter-Process Communication (IPC)

Protection and Security

Process Control

- Creation and termination of processes
- Suspension and resumption of processes
- Process synchronization and communication
- Handling deadlocks

File System Manipulation

CREATION AND DELETION OF FILES

READING AND WRITING FILES

DIRECTORY MANAGEMENT

FILE PERMISSIONS AND ACCESS CONTROL

Input/Output (I/O) Operations

DEVICE DRIVERS

BUFFERING AND CACHING

I/O SCHEDULING

ERROR DETECTION AND HANDLING

Inter-Process Communication (IPC)

Shared memory

Message passing

Pipes and sockets

Communication between cooperating processes

Resource Managment

CPU allocation and scheduling

Memory allocation and deallocation

Device management

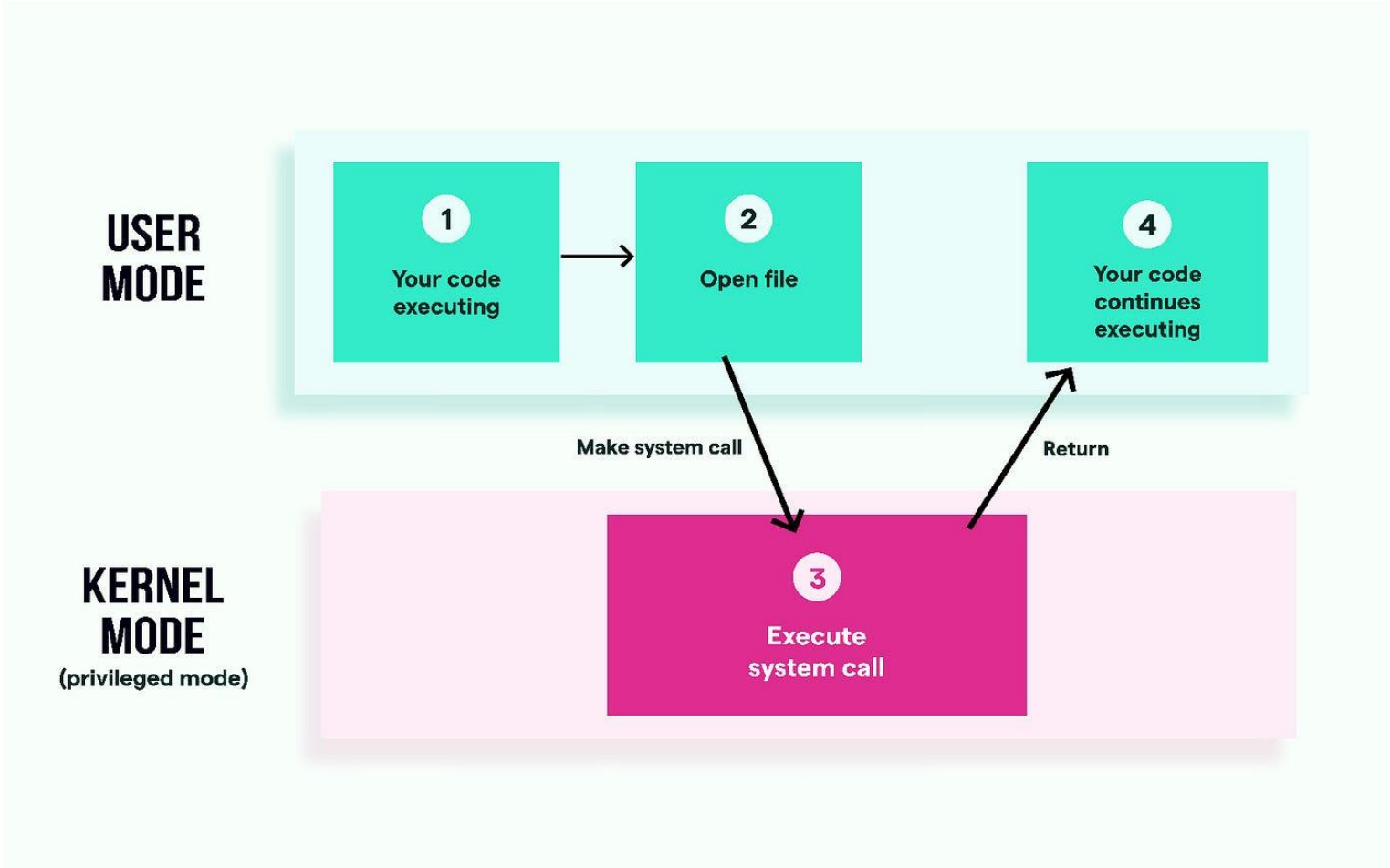
Storage and file resource management

How does a program ask the operating system for help? OR What happens behind the scenes when a program opens a file?

System Calls

Bridge between user application and OS.

Illustration



Confused between OS and kernel?

Kernel

The **Kernel** is the **core part of the Operating System**

It directly interacts with **hardware**

It controls **CPU, memory, and devices**

It runs in the **background**

The kernel is **not a separate software**, it is a **part of the OS**

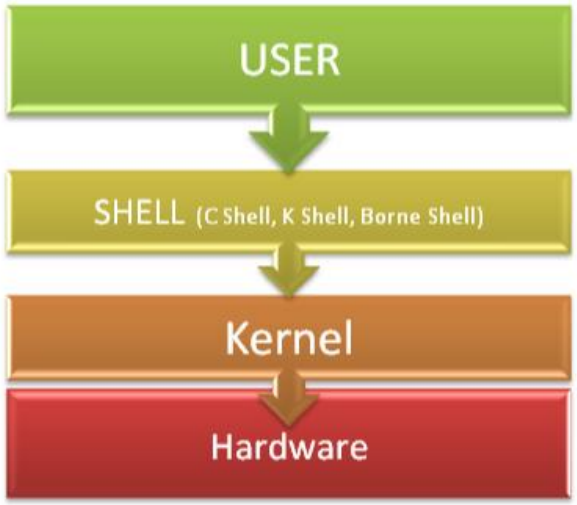
Key Idea

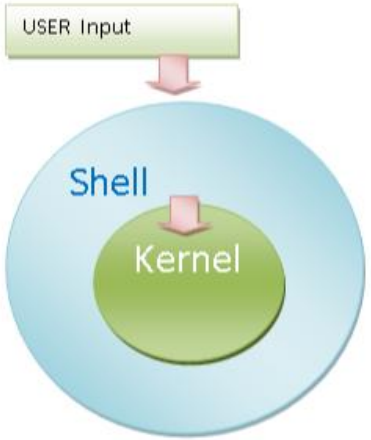
Operating System = Full system

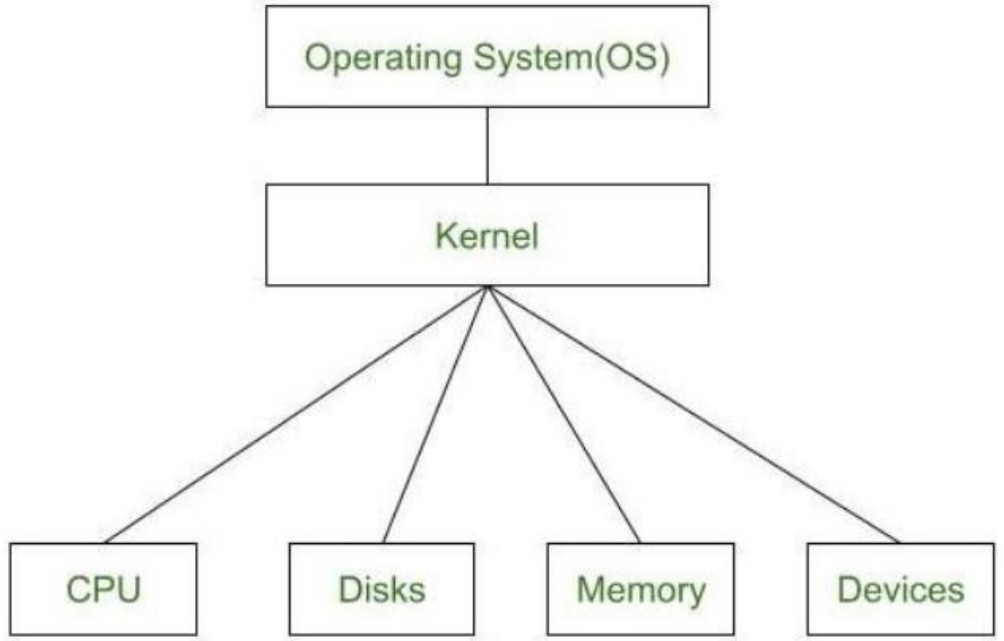
Kernel = Heart of the system

The OS cannot work without the kernel.

Illustration







User Interaction

Operating System	Kernel
• Users interact **directly** • Through GUI or Command Line	• Users **never interact directly** • Works silently in the background

Role and Responsibilities

Operating System

- Manages applications
- Provides user interface
- Handles files and security

Kernel

- Manages processes
- Allocates memory
- Controls hardware devices

Components

Operating System includes:

- Kernel
- File system
- User interface
- Utilities

Kernel includes:

- Process management
- Memory management
- Device control

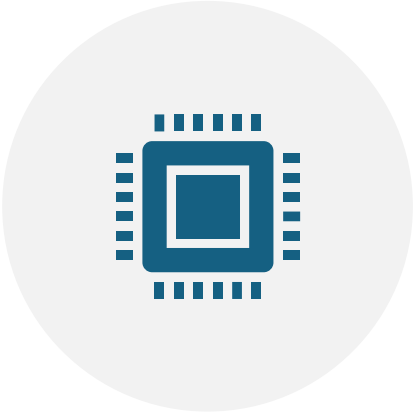
Comparison Table

Feature	Operating System	Kernel
Meaning	Complete system software	Core part of OS
User Interaction	Yes	No
Size	Large	Small
Hardware Access	Indirect	Direct
Failure Impact	Limited	System crash

Final Summary

THE **OPERATING SYSTEM** PROVIDES A COMPLETE ENVIRONMENT FOR USERS

THE **KERNEL** IS THE CORE THAT MAKES EVERYTHING WORK



OS DEPENDS ON KERNEL, BUT KERNEL IS HIDDEN FROM USERS

The Famous Confusion

Windows = Operating System

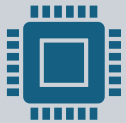
Windows NT = Kernel

BUT for Linux, it works slightly differently

How it works for **Windows**

Operating System: Windows (Windows 10, 11, etc.)

Kernel: Windows NT Kernel



Windows is an operating system built on top of the Windows NT kernel.

How it works for **Linux** (this is the key part)

Linux

- **Linux is technically the KERNEL**, not the full OS
- The **Operating System** is a **Linux Distribution**

Examples:

- Ubuntu
- Fedora
- Debian
- Red Hat

Why Linux sounds confusing (but isn’t)

People casually say:

- “I installed Linux”

What they really mean:

- “I installed a Linux-based operating system (like Ubuntu).”

Kernel and OS Types

Operating System Examples

- Windows (uses Windows NT kernel)
- Ubuntu, Fedora (Linux-based operating systems)
- macOS (uses XNU kernel)

Kernel Examples

- Windows NT
- Linux Kernel
- XNU (macOS)

Summary

Windows is an OS. Linux is a kernel. Ubuntu is an OS.

“Linux is the engine, Ubuntu is the car.”